

KFKA - Guia das páginas HTML para CSS e JS

Sep 22, 2026 · @André

Visão geral

O HTML das 15 páginas está pronto; falta o CSS (visual e responsividade) e o JS (dados, validação e navegação). Cada perfil tem uma página inicial e mais 3 páginas, todas com a mesma estrutura: cabeçalho com menu, trilha de navegação, seções com `<h2>` e rodapé.

```
public/
├─ css/           ← CSS (a criar)
├─ js/            ← JS (a criar)
└─ html/
    ├─ login/      login.html
    ├─ erro404/    404.html
    ├─ responsavel/ responsavel_inicio · responsavel_relatorios_filtro ·
responsavel_relatorio · responsavel_conta
    ├─ professor/  professor_inicio · professor_turmas ·
professor_acompanhamento · professor_conta
    └─ admin/      admin_inicio · admin_filtro_revisao · admin_revisar ·
admin_conta
```

O fluxo principal do sistema passa pelos três perfis:

```
flowchart LR
    A[Professor<br/>acompanhamento.html] -->|Enviar para revisão| B[Admin<br/>revisar.html]
    B -->|Devolver| A
    B -->|Publicar| C[Responsável<br/>relatorio.html]
    C -->|Registrar ciência| D[Professor vê o retorno<br/>acompanhamento.html]
```

Nenhuma página traz dados escritos no HTML: tudo começa com "Carregando.." ou "-" e o JS substitui pelo que vier da API.

Convenções de todas as páginas

Estas regras valem para as 15 páginas; resolva uma vez e reaproveite.

Elemento	O que é	O que fazer
<code>#nome-usuario</code>	Nome no cabeçalho	JS coloca o nome do usuário logado
<code>#botao-sair</code>	Botão "Sair"	JS encerra a sessão e volta para <code>/html/login/login.html</code>
<code>[aria-current="page"]</code>	Item do menu da página atual	CSS destaca esse item (use o atributo como seletor)
<code><nav aria-label="Você está em"></code>	Trilha de navegação	CSS: itens na mesma linha, separados por `>`
<code><tbody id="tabela-..."></code> com "Carregando..."	Tabela vazia esperando dados	JS troca pelas linhas; se vier vazio, "Nenhum resultado"; se a API falhar, a mensagem de erro
<code></code> ou <code><dd></code> com "Carregando..." ou "-"	Lista ou valor esperando dados	Mesmo tratamento das tabelas
<code>role="alert" + hidden</code>	Mensagem geral de um formulário	JS escreve o texto e tira o <code>hidden</code>
<code></code>	Erro de um campo	JS escreve o erro e põe <code>aria-invalid="true"</code> no campo; CSS pinta de vermelho
<code><small id="dica-..."></code> ou <code>regras-...</code>	Dica fixa embaixo do campo	Apenas CSS (texto menor, cinza)
<code>hidden</code> em seções inteiras	Blocos que só aparecem em certos casos	JS mostra quando o caso acontece

Formulários. Os formulários com `novalidate` são validados pelo JS:

`evento.preventDefault()`, confere os campos, mostra os erros e só então chama a API. Os de filtro (`role="search"`) também usam `preventDefault()` e atualizam a tabela sem recarregar a página.

Parâmetros de URL. Várias páginas recebem dados pela URL, como `?id=42` ou `?aluno=15`. Leia com `new URLSearchParams(window.location.search).get('id')`.

Segurança (XSS). Todo texto digitado por uma pessoa (descrição do professor, observação do responsável, motivo do admin, nomes) entra na página com `textContent`, nunca com `innerHTML`. Para montar linhas de tabela, crie os elementos com `document.createElement`.

Links. Sempre absolutos, começando com `/` (ex.: `/html/responsavel/responsavel_conta.html`), para funcionarem de qualquer pasta.

Valores dos `<select>`. Os `value` de status são iguais ao ENUM do `schema.sql` (`rascunho`, `enviado_revisao`, `em_revisao`, `publicado`, `devolvido`, `cancelado`) e os de bimestre vão de 1 a 4. Mande direto para a API, sem traduzir.

Login e 404

`html/login/login.html` é o formulário obrigatório da E1 ("validação antes do envio") e o melhor lugar para começar o JS. O endereço `localhost:3000/` já redireciona para ela.

id	Elemento	O JS faz
<code>form-login</code>	<code><form novalidate></code>	Escuta o <code>submit</code> , valida e chama <code>POST /api/login</code>
<code>email</code>	Campo de e-mail	Confere se está preenchido e no formato <code>nome@exemplo.com</code>
<code>senha</code>	Campo de senha	Confere se está preenchido
<code>erro-email</code> , <code>erro-senha</code>	Erros dos campos	Escreve a mensagem de cada campo
<code>mensagem-geral</code>	Mensagem geral (<code>hidden</code>)	Mostra "E-mail ou senha inválidos" ou erro de conexão

Depois do login, redirecione pelo perfil: `administrador` → `/html/admin/admin_inicio.html`, `professor` → `/html/professor/professor_inicio.html`, `responsavel` → `/html/responsavel/responsavel_inicio.html`. Durante o envio, desabilite o botão `botao-entrar` e troque o texto para "Entrando..." (é o "carregamento visível" da E1).

`html/erro404/404.html` não tem JS. O `server.js` já a entrega com status 404 para qualquer endereço inexistente, e `/api/...` inexistente responde JSON. Só falta o CSS.

Responsável

Fluxo: **Início** → **Relatórios** (lista com filtros) → **Relatório** (um em detalhe); **Minha conta** pelo menu. É a tela mais usada no celular, então é a prioridade do CSS responsivo.

Página (em <code>html/responsavel/</code>)	id	O JS faz
<code>responsavel_inicio.html</code>	<code>lista-alunos</code>	Um <code></code> por aluno (nome, turma) com link <code>/html/responsavel/responsavel_relatorios_filtro.html?aluno=ID</code>
<code>responsavel_inicio.html</code>	<code>tabela-recentes</code>	Últimos relatórios publicados; 5 colunas; link "Ver relatório" → <code>/html/responsavel/responsavel_relatorio.html?id=ID</code>
<code>responsavel_relatorios_filtro.html</code>	<code>form-filtros</code>	Filtros: <code>filtro-aluno</code> , <code>filtro-ano</code> , <code>filtro-bimestre</code> , <code>filtro-disciplina</code> . Ao abrir com <code>?aluno=</code> , pré-seleciona o aluno
<code>responsavel_relatorios_filtro.html</code>	<code>filtro-aluno</code> , <code>filtro-disciplina</code>	Criar os <code><option></code> a partir da API
<code>responsavel_relatorios_filtro.html</code>	<code>resumo-resultados</code>	"12 relatórios encontrados"
<code>responsavel_relatorios_filtro.html</code>	<code>tabela-relatorios</code>	7 colunas; link "Ver relatório" em cada linha. Dados de <code>GET /api/relatorios</code> (rota já pronta)
<code>responsavel_relatorios_filtro.html</code>	<code>pagina-anterior</code> , <code>pagina-atual</code> , <code>pagina-proxima</code>	Paginação: habilitar/desabilitar botões e escrever "Página 1 de 2" com <code>pagina</code> e <code>totalPaginas</code> da API
<code>responsavel_relatorio.html</code>	<code>mensagem-carregamento</code>	"Carregando..." → apagar; ou erro "Relatório não encontrado ou sem acesso"
<code>responsavel_relatorio.html</code>	<code>titulo-relatorio</code>	Ex.: "Matemática – 3º bimestre de 2026"
<code>responsavel_relatorio.html</code>	<code>relatorio-aluno</code> , <code>-turma</code> , <code>-disciplina</code> , <code>-professor</code> , <code>-bimestre</code> , <code>-media</code> , <code>-publicado-em</code>	Um valor em cada <code><dd></code>
<code>responsavel_relatorio.html</code>	<code>relatorio-descricao</code>	Texto do professor, com <code>textContent</code>
<code>responsavel_relatorio.html</code>	<code>relatorio-tags</code>	Um <code></code> por tag
<code>responsavel_relatorio.html</code>	<code>botao-pdf</code>	Baixa o PDF (RF09)
<code>responsavel_relatorio.html</code>	<code>status-ciencia</code>	"Ciência registrada em ..."; se já registrada, esconder o formulário
<code>responsavel_relatorio.html</code>	<code>form-ciencia</code>	Validar: <code>checkbox ciencia-confirmacao</code> marcado; <code>ciencia-observacao</code> até 500 caracteres. Erros em <code>erro-confirmacao</code> , <code>erro-observacao</code> ; mensagem em <code>mensagem-ciencia</code>
<code>responsavel_conta.html</code>	<code>conta-nome</code> , <code>conta-email</code> , <code>conta-alunos</code>	Dados da conta e lista de alunos vinculados
<code>responsavel_conta.html</code>	<code>form-senha</code>	Ver "Formulário de senha" abaixo

Formulário de senha (igual nas três contas): campos `senha-atual`, `senha-nova`, `senha-confirmacao`; erros `erro-senha-atual`, `erro-senha-nova`, `erro-senha-confirmacao`; mensagem `mensagem-senha`. Regras: todos obrigatórios, nova senha com mínimo de 8 caracteres (12 no admin), com letra e número, e confirmação igual à nova. É o terceiro formulário com validação, se sobrar tempo.

Professor

Fluxo: **Início** → **Minhas turmas** (escolhe turma e vê alunos) → **Acompanhamento** (registra ou edita); **Minha conta** pelo menu. A página `professor_acompanhamento.html` é a mais complexa do projeto.

Página (em <code>html/professor/</code>)	id	O JS faz
<code>professor_inicio.html</code>	<code>periodo-digitacao</code>	Ex.: "3º bimestre aberto até 30/09/2026 às 23:59"
<code>professor_inicio.html</code>	<code>lista-turmas</code>	Um <code></code> por turma + disciplina, com link <code>/html/professor/professor_turmas.html?turma=ID&disciplina=ID</code>
<code>professor_inicio.html</code>	<code>form-filtro-status</code> , <code>filtro-status</code>	Filtra a tabela por status
<code>professor_inicio.html</code>	<code>tabela-acompanhamentos</code>	7 colunas; link "Abrir" → <code>/html/professor/professor_acompanhamento.html?id=ID</code>
<code>professor_turmas.html</code>	<code>form-turma</code> , <code>selecao-turma</code> , <code>selecao-bimestre</code>	Criar os <code><option></code> de turma; ao enviar, buscar os alunos. Pré-selecionar se vier <code>?turma=&disciplina=</code>
<code>professor_turmas.html</code>	<code>resumo-turma</code>	Ex.: "5º A - Matemática · 3º bimestre · 18 de 25 registrados"
<code>professor_turmas.html</code>	<code>tabela-alunos</code>	4 colunas. Sem registro: link "Registrar" com <code>?aluno=&turma=&disciplina=&bimestre=</code> . Com registro: "Abrir" com <code>?id=</code>
<code>professor_acompanhamento.html</code>	<code>titulo-pagina</code>	"Registrar acompanhamento" ou "Editar acompanhamento"
<code>professor_acompanhamento.html</code>	<code>status-acompanhamento</code> , <code>periodo-acompanhamento</code>	Status atual e período; fora do período, bloquear o formulário
<code>professor_acompanhamento.html</code>	<code>bloco-devolucao</code> , <code>motivo-devolucao</code>	Mostrar só se o status for <code>devolvido</code> , com o motivo do admin
<code>professor_acompanhamento.html</code>	<code>form-acompanhamento</code>	Campos <code>campo-turma</code> , <code>campo-aluno</code> , <code>campo-bimestre</code> , <code>campo-descricao</code> , <code>campo-media</code> ; erros <code>erro-*</code> de cada um; mensagem <code>mensagem-acompanhamento</code>
<code>professor_acompanhamento.html</code>	<code>lista-tags</code>	Criar um checkbox + label por tag (<code>name="tags"</code> , <code>value = id da tag</code>)
<code>professor_acompanhamento.html</code>	Botões <code>name="acao"</code>	Os dois são submit: <code>evento.submitter.value</code> diz se foi <code>rascunho</code> ou <code>enviar</code>
<code>professor_acompanhamento.html</code>	<code>secao-retorno</code> , <code>retorno-ciencia</code> , <code>retorno-observacao</code>	Mostrar só se <code>publicado</code> , com a ciência e a observação do responsável
<code>professor_conta.html</code>	<code>conta-nome</code> , <code>conta-email</code> , <code>conta-turmas</code> , <code>form-senha</code>	Igual à conta do responsável, com turmas no lugar de alunos

A página de acompanhamento abre de três jeitos. Sem parâmetros: formulário vazio. Com `?aluno=&turma=&disciplina=&bimestre=` : campos pré-preenchidos. Com `?id=` : busca o registro e preenche tudo; se o status não for `rascunho` nem `devolvido` , o formulário fica só para leitura.

Validação: turma, aluno, bimestre e descrição obrigatórios; descrição até 2000 caracteres; média de 0 a 10 com até uma casa decimal (mesma regra do `CHECK` no banco).

Administrador

Fluxo: **Início** → **Revisão** (lista com filtros) → **Revisar** (um registro e a decisão); **Minha conta** pelo menu. Cadastros, bimestres e relatórios ficam para a E2.

Página (em <code>html/admin/</code>)	id	O JS faz
<code>admin_inicio.html</code>	<code>resumo-bimestre</code> , <code>resumo-</code> <code>aguardando</code> , <code>resumo-alunos</code> , <code>resumo-</code> <code>professores</code>	Um número ou texto em cada <code><dd></code>
<code>admin_inicio.html</code>	<code>tabela-revisao</code>	7 colunas; link "Revisar" → <code>/html/admin/admin_revisar.html?id=ID</code>
<code>admin_filtro_revisao.html</code>	<code>form-filtros</code>	Filtros <code>filtro-status</code> (já vem em <code>enviado_revisao</code>), <code>filtro-turma</code> , <code>filtro-disciplina</code> , <code>filtro-professor</code> , <code>filtro-bimestre</code> , <code>filtro-aluno</code> (busca por nome)
<code>admin_filtro_revisao.html</code>	<code>filtro-turma</code> , <code>filtro-</code> <code>disciplina</code> , <code>filtro-</code> <code>professor</code>	Criar os <code><option></code> a partir da API
<code>admin_filtro_revisao.html</code>	<code>resumo-</code> <code>resultados</code> , <code>tabela-</code> <code>acompanhamentos</code>	Resumo e tabela de 8 colunas; link "Revisar" (ou "Ver" se publicado/cancelado)
<code>admin_filtro_revisao.html</code>	<code>pagina-anterior</code> , <code>pagina-atual</code> , <code>pagina-proxima</code>	Paginação, igual à do responsável
<code>admin_revisar.html</code>	<code>mensagem-</code> <code>carregamento</code> , <code>titulo-pagina</code>	Carregamento/erro; título como "Ana Souza – Matemática – 3º bimestre"

Página (em html/admin/)	id	O JS faz
admin_revisar.html	revisar-aluno , - turma , - disciplina , - professor , - bimestre , - status	Um valor em cada <dd>
admin_revisar.html	form-conteudo	Campos campo-descricao , campo-media , lista-tags (checkboxes já marcados), campo-justificativa ; erros erro-* ; mensagem mensagem-conteudo
admin_revisar.html	form-decisao	Rádios decisao-publicar , decisao-devolver , decisao-cancelar (name="decisao"); campo-motivo ; erros erro-decisao , erro-motivo ; mensagem mensagem-decisao
admin_revisar.html	tabela-historico	5 colunas, do mais recente para o mais antigo
admin_conta.html	conta-nome , conta-email , form-senha	Igual às outras contas; senha nova com mínimo de 12 caracteres

Validação de form-conteudo : se a descrição, a média ou as tags mudaram em relação ao que o professor mandou, a justificativa é obrigatória. Guarde os valores originais numa variável quando a página carregar e compare no envio.

Validação de form-decisao : uma opção precisa estar marcada; campo-motivo é obrigatório para devolvido e cancelado . Antes de publicar, vale um confirm() , porque a família passa a ver o relatório.

Rotas da API sugeridas

São sugestões para o front e o back combinarem os mesmos endereços. Hoje o server.js tem /teste-db , a 404 e GET /api/relatorios , já pronta e testada; as outras ainda não existem. Todas respondem JSON e, em erro, { "erro": "mensagem" } .

Método e rota	Usada em	Devolve
POST /api/login	login	{ usuario: { id, nome, perfil } } ou 401
GET /api/responsavel/alunos	responsável: início, relatórios, conta	Alunos vinculados (id, nome, turma)
GET /api/relatorios?aluno=&ano_letivo=&bimestre=&disciplina=&pagina= (pronta)	responsável: início, relatórios	Lista paginada da view vw_relatorio_publicado
GET /api/responsavel/relatorios/:id	responsável: relatório	Um relatório + tags + ciência
POST /api/responsavel/relatorios/:id/ciencia	responsável: relatório	Registra ciência e observação
GET /api/professor/turmas	professor: início, turmas, acompanhamento, conta	Pares turma + disciplina do professor
GET /api/professor/turmas/:turma/alunos?disciplina=&bimestre=	professor: turmas	Alunos + status do acompanhamento
GET /api/professor/acompanhamentos?status=	professor: início	Acompanhamentos do professor
GET/POST/PUT /api/professor/acompanhamentos/:id	professor: acompanhamento	Ler, criar, editar (acao : rascunho ou enviar)
GET /api/tags	professor e admin	Tags cadastradas
GET /api/admin/resumo	admin: início	Números do resumo
GET /api/admin/acompanhamentos?status=&turma=&...&pagina=	admin: início, revisão	Lista paginada
GET/PUT /api/admin/acompanhamentos/:id	admin: revisar	Ler e alterar conteúdo (com justificativa)
POST /api/admin/acompanhamentos/:id/decisao	admin: revisar	Publicar, devolver ou cancelar (com motivo)
GET /api/admin/acompanhamentos/:id/historico	admin: revisar	Linhas da tabela historico
GET /api/conta e PUT /api/conta/senha	as três contas	Dados da conta; troca de senha

Para a E1, basta uma rota real alimentando uma tabela (a E1 aceita API real ou mockada). Ela já existe: GET /api/relatorios lê a view vw_relatorio_publicado e devolve { pagina, porPagina, total, totalPaginas, relatorios: [...] }. Para testar sem

front, abra `localhost:3000/api/relatorios?aluno=1&bimestre=3` com o MySQL rodando e o `seed.sql` carregado.

Segurança (E2): o backend precisa conferir o perfil e o vínculo em toda rota. Exemplo: em `/api/responsavel/relatorios/:id`, confirmar que o relatório é de um aluno daquele responsável e está publicado; senão, 403. A tabela `historico` ainda não tem coluna para o motivo e a justificativa; avisar quem cuida do banco.

Detalhes da API para o JS

- **Datas:** `publicadoEm` chega em UTC, como `"2026-09-18T18:00:00.000Z"`. Para mostrar, use `new Date(r.publicadoEm).toLocaleDateString('pt-BR')` → `"18/09/2026"`.
- **Médias:** `media` chega como número (`9`, `8.5`). Para mostrar com vírgula e uma casa, use `r.media.toLocaleString('pt-BR', { minimumFractionDigits: 1 })` → `"9,0"`, `"8,5"`.
- **Sem MySQL** (por exemplo, na rede da faculdade), a rota responde **500** com `{ "erro": "Não foi possível carregar os relatórios. Tente novamente." }`. Mostre esse texto na tabela: é o "tratamento visível de erro" da E1.
- **Filtro inválido** (como `bimestre=9`) responde **400** com `{ "erro": "..." }`.

Contrato combinado do login (rota ainda a fazer)

- **Pedido:** `POST /api/login`, `Content-Type: application/json`, corpo `{ "email": "...", "senha": "..." }`.
- **Sucesso:** 200 e `{ "usuario": { "id": 2, "nome": "Carla Lima", "perfil": "professor" } }`.
- **Erros:** 400 `{ "erro": "Informe e-mail e senha." }`, 401 `{ "erro": "E-mail ou senha inválidos." }`, 500 `{ "erro": "..." }`.

Usuários de teste do `seed.sql`, todos com a senha `senha123`:

Perfil	E-mail
Administrador	coordenacao@escola-teste.com
Professor	carla.lima@escola-teste.com
Responsável	mariana.souza@exemplo.com

Prioridades até a Entrega 1

A E1 vence em `Sep 25, 2026` às 23:59, e cada dia de atraso custa 25% da nota. Faça nesta ordem; os itens marcados com (E1) são exigência direta do documento.

- ☐ **CSS base** (`css/style.css` , um arquivo para todas as páginas): variáveis de cor do Figma, cabeçalho, menu, formulários, tabelas e mensagens de erro (E1: "CSS organizado")
- ☐ **Responsividade**: mobile first; menu que quebra em linhas no celular; tabelas dentro de um bloco com `overflow-x: auto` para não estourar a tela (E1: "responsiva")
- ☐ **JS do login**: validação antes do envio e `fetch` para `/api/login` (E1: "formulário com validação")
- ☐ **Uma tabela com `fetch`** mostrando "Carregando..." e erro visível (E1: "consumo assíncrono"); sugestão: `tabela-relatorios` do responsável
- ☐ **Botão Sair** e `#nome-usuario` em todas as páginas
- ☐ **Commits de todos os integrantes** no repositório (E1)

Dicas de CSS para esta estrutura:

- Ligue o CSS em todas as páginas com `<link rel="stylesheet" href="/css/style.css">` no `<head>` ; hoje só o HTML existe.
- Estilize pelas tags e atributos que já existem (`header nav` , `[aria-current="page"]` , `[aria-invalid="true"]` , `table` , `dl`), assim quase não é preciso mexer no HTML.
- `<dl>` com `<div>` dentro vira uma grade de "rótulo: valor" com `display: grid` .
- `[hidden]` precisa continuar escondido: se algum `display` sobrescrever, adicione `[hidden] { display: none !important; }` .
- Confira o contraste das cores (texto escuro em fundo claro); acessibilidade é requisito do PI.